RESUME ANALYSER

AI-POWERED RESUME AND JOB DESCRIPTION ANALYSIS SYSTEM

MINI PROJECT REPORT

Submitted in partial fulfilment of the requirements for the award of

[Degree / Programme Name]

Submitted by

[Student Name] | [Roll Number]

Under the guidance of

[Guide Name and Designation]

[Department Name] | [College / University Name]

Academic Year 2026-27

Live application: https://mini-project-six-black.vercel.app/

CERTIFICATE

This is to certify that the mini project report entitled “Resume Analyser” is a bonafide record of work carried out by the student named below, under the supervision of the undersigned, in partial fulfilment of the academic requirements of [Degree / Programme Name] at [College / University Name].

STUDENT DETAILS	INFORMATION
Name	[Student Name]
Roll / Registration No.	[Roll Number]
Department	[Department Name]
Academic year	2026-27

The project describes a web application that accepts a target job description and a candidate profile and then creates AI-assisted preparation material. Technical statements in this report are based on inspected source files and the deployed application at the time of preparation. The project is submitted for academic review and remains subject to the institution’s normal evaluation process.

The undersigned certify that the student has completed the project work to the best of their knowledge and has presented the work in this report in a suitable academic format.

PROJECT GUIDE	HEAD OF DEPARTMENT	EXTERNAL EXAMINER
Signature: _____	Signature: _____	Signature: _____
Name: [Guide Name]	Name: [HOD Name]	Name: _____
Date: _____	Date: _____	Date: _____

DECLARATION

I hereby declare that the mini project entitled “Resume Analyser” is a record of work completed for academic submission. The application and report describe a software project that compares a candidate profile with a target job description and presents AI-assisted preparation guidance.

I further declare that external documentation and project resources used in preparing this report are identified in the references. No confidential credentials or private user data are intentionally reproduced. Illustrative examples are not represented as measured results unless explicitly identified as observed project behavior.

The deployed application and its underlying source code may change over time. This report reflects the inspected project version as of 28 October 2026. Generated recommendations are advisory and do not guarantee interview selection, employment or an objective evaluation of candidate ability.

DECLARANT	DETAIL
Student name	[Student Name]
Roll number	[Roll Number]
Place	[City]
Date	[Submission Date]
Signature	_____

ACKNOWLEDGEMENT

I express my sincere gratitude to [Project Guide Name], [Designation], for guidance, feedback and encouragement during the planning, implementation and documentation of this mini project. Their suggestions helped me review the project from both a technical and an academic perspective.

I thank the Head of the Department, faculty members and [College / University Name] for providing the academic environment and resources needed to complete the work. I am also grateful to classmates, friends and family members who offered suggestions and helped review the application.

The project uses open-source frameworks and developer tools, including React, Vite, Node.js, Express, MongoDB, Mongoose, Zod and Puppeteer, together with the Google Gemini API. I acknowledge the maintainers and documentation authors of these technologies in the reference section.

Finally, I acknowledge the availability of the deployed application and source repository, which make it possible to review the product behavior and implementation alongside this report.

> **STUDENT NOTE:** Replace all bracketed details with accurate information before printing or submission.

ABSTRACT

Resume Analyser is a web-based mini project designed to help a candidate prepare for a specific job opportunity. A user supplies a job description and one profile source: a resume upload or a short self-description. The application sends these inputs to a server-side AI service, which asks Google Gemini to produce structured preparation content. The resulting report can include a role title, a match score, technical questions with answer guidance, behavioral questions, skill gaps with severity, and a day-wise preparation plan.

The system is implemented as a React and Vite frontend with an Express and Node.js API. MongoDB stores account and report data through Mongoose models. Account registration and login use password hashing and a signed JSON Web Token stored in an HTTP-only cookie; interview-report endpoints require authentication. The backend also provides PDF generation using Puppeteer and parses uploaded PDF text using pdf-parse. The application is deployed with a Vercel frontend and a Render API and depends on an external MongoDB service and Gemini API credentials.

The project demonstrates full-stack integration, REST API design, data persistence, cookie-based authentication, file handling, schema-shaped generative AI output and document export. Generated advice is not independently verified and depends on the quality of the submitted profile, job description and model response. Source inspection also shows that DOCX upload is advertised in the interface while the backend extraction path uses pdf-parse; reliable DOCX extraction is therefore a verification and improvement item.

KEYWORDS: Resume analysis; generative AI; Gemini; React; Express; MongoDB; interview preparation; REST API.

TABLE OF CONTENTS

The report is organized from academic front matter to project context, design, implementation, evaluation and appendices. The table below includes a serial number and the Roman numeral page on which each section begins. Page references are populated from the final Word pagination.

SERIAL NO.	SECTION / HEADING	PAGE NO.
01	Certificate	ii
02	Declaration	iii
03	Acknowledgement	iv
04	Abstract	v
05	Table of Contents	vi
06	Table of Contents (continued)	vii
07	List of Figures, Tables and Abbreviations	viii
08	Chapter 1 - Introduction	ix
09	Chapter 1 - Problem, Aim, Objectives and Scope	x
10	Chapter 2 - Background and Proposed System	xii
11	Chapter 3 - Requirements Analysis	xiii
12	Chapter 3 - Feasibility and Development Method	xiv
13	Chapter 4 - System Architecture	xv
14	Chapter 4 - Technology Stack	xvi
15	Chapter 5 - Data Design and Report Model	xvii

TABLE OF CONTENTS (CONTINUED)

Continued contents and appendices.

SERIAL NO.	SECTION / HEADING	PAGE NO.
16	Chapter 5 - Workflow and Use Cases	xviii
17	Chapter 6 - AI Methodology	xix
18	Chapter 6 - API and Security Design	xx
19	Chapter 7 - User Interface and Accessibility	xxi
20	Application Screenshots - Home	xxii
21	Application Screenshots - Authentication	xxiv
22	Chapter 8 - Testing and Quality Plan	xxv
23	Chapter 8 - Deployment and Operations	xxvi
24	Chapter 9 - Results and Discussion	xxvii
25	Chapter 9 - Limitations and Future Scope	xxvii
26	Chapter 10 - Conclusion	xxix
27	References	xxx
28	Appendix A - Technical Setup and Configuration	xxxi
29	Appendix B - API Reference and Response Schema	xxxii

LIST OF FIGURES, TABLES AND ABBREVIATIONS

List of figures

FIGURE NO.	SCREENSHOT / CAPTION	PAGE NO.
Figure 1	Resume Analyser home page in dark theme.	xxiii
Figure 2	Resume Analyser home page in light theme.	xxiii
Figure 3	Sign-in page with email and password fields.	xxiv
Figure 4	Account-creation page with username, email and password fields.	xxiv

List of tables

TABLE NO.	TITLE	PAGE NO.
1.1	Objectives and measurable indicators	x
3.1	Functional requirements	xiii
3.2	Non-functional requirements	xiii
4.1	Application architecture components	xv
4.2	Technology stack	xvi
5.1	Core data entities	xvii
6.1	API endpoint catalogue	xx
8.1	Verification test cases	xxv
A.1	Environment configuration inventory	xxvi

Abbreviations

TERM	MEANING
AI	Artificial Intelligence
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete

HTML	HyperText Markup Language
HTTP / HTTPS	HyperText Transfer Protocol / Secure
JWT	JSON Web Token
MERN	MongoDB, Express, React, Node.js
PDF	Portable Document Format
REST	Representational State Transfer
UI / UX	User Interface / User Experience
URL	Uniform Resource Locator

CHAPTER 1 - INTRODUCTION

1.1 Introduction

A resume communicates a candidate's experience, while a job description states what an employer expects for a role. Comparing the two and deciding how to prepare can take time, particularly when a candidate must identify relevant gaps, anticipate questions and plan practice sessions. Resume Analyser brings these tasks into one web workflow.

The application accepts a target job description and a candidate profile. A backend service assembles the provided context and calls a generative AI model. The response is structured so that the interface can present an alignment score, question sets, skill-gap observations and a day-by-day preparation plan. A signed-in user can revisit saved reports and request PDF outputs.

1.2 Project context

The project is a practical demonstration of a MERN-style web application enhanced with a hosted generative AI service. It combines client-side form handling, API routing, authentication, database persistence, document parsing and PDF rendering. Its current interface uses the product label "InterviewAI"; this report uses the requested project title "Resume Analyser".

1.3 Intended users

- Students preparing for internships, placements or graduate roles.
- Early-career applicants who want to organize practice around a job posting.

- Candidates seeking question prompts and an actionable preparation sequence.

RESPONSIBLE USE: The tool supports preparation; it is not an applicant-screening system and should not make hiring decisions.

CHAPTER 1 - PROBLEM, AIM, OBJECTIVES AND SCOPE

1.4 Problem statement

Candidates often receive a long job description but lack a concise method to map its requirements to their experience. Generic interview resources may not reflect the target role, while informal preparation can omit prioritization and a clear sequence of practice. A useful preparation workflow needs to connect role expectations to a candidate's own context and present the result in a form that can be acted upon.

1.5 Aim

To design and implement a web application that uses a candidate profile and a job description to generate structured, role-specific interview preparation guidance.

1.6 Objectives

NO.	OBJECTIVE	SUCCESS INDICATOR
1	Accept role and candidate context.	Job description plus resume or self-description can be submitted.
2	Generate structured guidance.	Report includes configured score, questions, gaps and plan sections.
3	Protect personal reports.	Protected endpoints associate report access with the signed-in user.
4	Enable review and export.	A saved report can be reopened and PDF outputs can be requested.

1.7 Scope and assumptions

The scope includes account flows, profile input, report generation, report history and PDF export. It assumes network access to the API, database and Gemini service, valid server configuration and user review of recommendations. Recruitment integrations,

verified job-market data, guaranteed scoring accuracy and professional career counseling are outside the current scope.

1.8 Acceptance criteria

The minimum acceptance criteria are that a new user can register and sign in; the authenticated user can submit a job description with a resume or profile summary; the service either returns a structured report or a clear error; saved reports are available only through the intended account flow; and the user can review or export the resulting preparation material. These criteria describe observable behavior, not a claim that the generated score or advice is objectively correct. Each should be exercised with valid, invalid and boundary inputs before a release.

INPUT CONSTRAINTS: The interface indicates a 5,000-character job-description limit and a 3 MB resume-upload limit.

CHAPTER 2 - BACKGROUND AND PROPOSED SYSTEM

2.1 Background

Generative AI systems can transform natural-language context into structured content. In Resume Analyser, the model is instructed to act as an interview coach and return JSON matching a defined schema. Schema-shaped output helps the frontend render predictable sections, although it does not prove that the advice is factual, unbiased or suitable for every candidate.

2.2 Existing preparation approaches

- Manual comparison of resume and role requirements can be thorough but takes effort.
- Static question banks are reusable but may not reflect an individual candidate.
- Spreadsheets can organize tasks but require users to perform their own analysis.
- General-purpose chat tools can return inconsistent response formats and require users to structure the output.

2.3 Proposed solution

Resume Analyser combines profile input, a job target and AI-generated structure. It provides tailored technical and behavioral prompts, answer guidance, gap-severity labels, a match score and a day-by-day preparation plan. Reports are saved against an account and can be exported for later study.

2.4 Differentiating characteristics

CAPABILITY	PROJECT APPROACH
Context	Both the target job and candidate background are included.
Structure	A Zod response shape is converted to a JSON schema for Gemini.
Continuity	A signed-in user can retrieve report history.
Export	The backend can render a plan PDF and generate a resume PDF.
Transparency	The score and recommendations are AI-assisted, not verified.

The system does not independently verify work history, skill proficiency, job validity or model-generated facts.

CHAPTER 3 - REQUIREMENTS ANALYSIS

3.1 Functional requirements

ID	REQUIREMENT	PRIORITY
FR-01	Register an account and store a password hash.	High
FR-02	Authenticate a user and maintain a cookie-based session.	High
FR-03	Accept a job description and resume upload or self-description.	High
FR-04	Generate a structured AI interview report.	High
FR-05	Save reports and retrieve a user's report history.	High
FR-06	Retrieve a specific report subject to authentication.	High
FR-07	Generate PDF output for an interview plan and resume.	Medium
FR-08	Allow switching between light and dark themes.	Medium

3.2 Non-functional requirements

AREA	REQUIREMENT / DESIGN CONSIDERATION
Usability	Responsive layout, form guidance and understandable error feedback.
Security	Password hashing, signed JWT, HTTP-only cookie and protected routes.
Privacy	Resume and report data are sensitive; do not log submitted content.
Reliability	AI, database and PDF operations can fail and need explicit handling.
Performance	AI generation can take time; show progress and status.
Maintainability	Separate frontend features, backend routes, models and AI service.
Compatibility	Modern browser and Node.js runtime with deployment configuration.

3.3 Constraints

The service depends on third-party availability, API quotas and correctly configured secrets. AI output can vary between runs. Uploaded-file processing has a configured 3

MB limit, and the current backend parsing path is PDF-oriented. The inspected frontend package defines build and lint scripts but no dedicated automated test script.

Acceptance demonstration: sign in, generate a report with valid inputs, retrieve the saved report and download a PDF.

CHAPTER 3 - FEASIBILITY AND DEVELOPMENT METHOD

3.4 Technical feasibility

The selected tools support a conventional browser-to-REST architecture. React and Vite support frontend development and bundling; Express exposes API routes; Mongoose provides MongoDB models; the Gemini SDK handles model requests; Puppeteer and pdf-parse support document processing. The app is deployed on Vercel and Render, with separate database and AI dependencies.

3.5 Operational feasibility

The workflow is designed for ordinary applicants: paste a job description, choose a profile source and review an organized report. Users need an account for report functionality. Clear notices are appropriate for sensitive resume data, third-party AI processing and generated advice. A short demonstration can show the home page, authentication, generated report and download flow.

3.6 Economic feasibility

The project uses widely available developer tooling and hosted services. Operational costs may arise from AI usage, database hosting, web hosting, file processing and bandwidth. Actual cost depends on provider plans and request volume; this report does not assume free or unlimited production capacity. A college demonstration can use test data and controlled requests.

3.7 Development method

An iterative workflow is suitable: inspect requirements, implement one end-to-end feature, validate UI and API, then improve usability and deployment. The final evaluation should record the date, test data, environment, outcomes and failures rather than relying on informal impressions.

PHASE	OUTPUT
Analysis	Feature and constraint list

Design	Architecture, data and API contracts
Implementation	Frontend, API, model and AI integration
Verification	Build plus repeatable scenario tests
Delivery	Deployment, report and demonstration

A staged workflow limits rework: agree on the scope first, implement an observable user journey, test it with representative data, and then record both successful and unsuccessful outcomes.

CHAPTER 4 - SYSTEM ARCHITECTURE

4.1 Logical architecture

LAYER	COMPONENTS	RESPONSIBILITY
Presentation	React; Vite; SCSS	Collect input, manage session UI, display reports and start downloads.
API	Node.js; Express routers/controllers	Authenticate users, coordinate services and return responses.
AI service	Gemini SDK; Zod schema	Request structured candidate-specific report or resume content.
Persistence	MongoDB; Mongoose	Store account, blacklist and interview-report data.
Documents	pdf-parse; Puppeteer	Extract uploaded PDF text and render HTML into PDFs.
Hosting	Vercel; Render	Serve browser application and backend API separately.

4.2 Request and response path

The browser sends HTTPS requests to the backend using Axios with credentials enabled. Protected requests include the authentication cookie. Express routes run authentication middleware, invoke controllers, and call Mongoose or the AI service as required. The controller returns JSON or a generated PDF. The database and AI model are reached from the server, so their credentials should not be exposed in the browser.

4.3 Logical communication map

ACTOR / SERVICE	COMMUNICATION	ACTOR / SERVICE
Browser (React)	HTTPS REST + cookie	Express API
Express API	Gemini SDK request	Google Gemini API
Express API	Mongoose queries	MongoDB
Express API	HTML to PDF	Puppeteer renderer
Browser	Static assets	Vercel hosting
Browser / API	Cross-origin HTTPS	Vercel / Render

The UI and API are independently deployed. CORS and cross-site cookie settings are therefore part of the production request path.

CHAPTER 4 - TECHNOLOGY STACK

4.4 Frontend

React 19 provides component-based UI. Vite 7 runs the development server and builds production assets. SCSS styles the application, including light and dark themes. Axios provides HTTP calls; React Router provides navigation; React Hot Toast presents notifications. The frontend feature folders separate authentication and interview-preparation screens.

4.5 Backend and data

Node.js runs the server and Express 5 defines middleware and REST routes. MongoDB stores data through Mongoose 9 schemas. bcryptjs hashes passwords; jsonwebtoken signs session claims; cookie-parser reads cookies; CORS configures cross-origin API access; Multer holds uploaded files in memory.

4.6 AI and document tools

The @google/genai SDK calls the model identified in source as gemini-3.1-flash-lite. Zod describes expected report structure and zod-to-json-schema prepares the response schema. pdf-parse extracts PDF text. Puppeteer launches a headless browser to render HTML into PDF documents.

AREA	TECHNOLOGY	ROLE
UI	React, Vite, SCSS, Axios	Browser application

Routing / feedback	React Router, React Hot Toast	Navigation and notifications
API	Node.js, Express, CORS	REST service
Data	MongoDB, Mongoose	Persistence
Identity	bcryptjs, jsonwebtoken, cookies	Authentication mechanism
AI / schema	Gemini SDK, Zod	Structured generation
Files / export	Multer, pdf-parse, Puppeteer	Upload, extraction and PDF rendering
Hosting	Vercel, Render	Frontend and API deployment

CHAPTER 5 - DATA DESIGN AND REPORT MODEL

5.1 Core entities

ENTITY	PURPOSE	RELATIONSHIP
User	Account with username, email and stored password hash.	Owns zero or more reports.
InterviewReport	Source context and generated preparation sections.	References the owning user.
Blacklist entry	Records a token invalidated during logout.	Checked by authentication middleware.

5.2 Interview-report structure

- Role title and job-description context.
- Profile context from extracted resume text and/or self-description.
- Match score bounded in the model schema from 0 to 100.
- Technical and behavioral items contain a question, intention and answer guidance.
- Skill gaps include a skill and low, medium or high severity.
- Preparation-plan items contain a day, focus and task list.
- Mongoose timestamps track report creation and update.

5.3 Data access and retention

Report detail and history controllers query with the authenticated user identity, and history omits several large fields from list responses. The resume PDF controller uses a report ID lookup; its route is protected by authentication, but the inspected controller lookup itself is not visibly scoped by user ownership. This should be reviewed before production use.

Resume text and job descriptions can contain personal or confidential material. A production deployment should document retention, deletion, access, encryption, third-party AI processing, backups and incident response. The schema is a technical model, not a complete privacy policy.

Model definitions reviewed: Backend/src/models/user.model.js and Backend/src/models/interviewReport.model.js.

CHAPTER 5 - WORKFLOW AND USE CASES

5.4 Primary user workflow

STEP	USER ACTION	SYSTEM RESPONSE
1	Register or sign in.	Hash password on registration; issue JWT cookie after login.
2	Enter target job description.	Frontend shows guidance and a character counter.
3	Upload a resume or write a profile summary.	Multer accepts a file in memory; controller extracts PDF text.
4	Request an interview strategy.	API checks required inputs and calls the AI service.
5	Review generated results.	Report is saved and returned to the user interface.
6	Open history or a selected report.	Authenticated endpoints retrieve reports.
7	Download an output.	Backend returns a generated PDF attachment.

5.5 Use-case summary

USE CASE	ACTOR	PRECONDITION	OUTCOME
Create account	Visitor	Valid registration values	Account created and session issued.
Generate report	Signed-in user	Role plus one profile source	AI report saved and displayed.
View history	Signed-in user	At least one saved report	List of report summaries returned.
Download plan PDF	Signed-in user	Owned report exists	PDF attachment returned.
Log out	Signed-in user	Session cookie present	Cookie cleared and token blacklisted.

ERROR SCENARIOS: Missing role/profile results in 400; missing or invalid session in 401; missing report in 404; generation failures may return 500.

CHAPTER 6 - AI METHODOLOGY

6.1 Input preparation

The report-generation controller receives a job description and either a file or self-description. For an uploaded file, the backend attempts PDF text extraction and passes extracted text to the AI service. A self-description can be supplied directly. The service interpolates these values into a prompt that asks for an interview report using the configured JSON shape.

6.2 Structured output schema

FIELD	EXPECTED CONTENT
title	Job title or report heading.
matchScore	Numeric alignment estimate from 0 to 100.
technicalQuestions	Question, interviewer intention and suggested answer.
behavioralQuestions	Question, intention and suggested answer.
skillGaps	Skill plus low / medium / high severity.

preparationPlan	Day number, focus and actionable tasks.

6.3 Model interaction

The backend calls Google Gemini using gemini-3.1-flash-lite and requests application/json with a schema generated from Zod. The service parses the returned text as JSON. This constrains response format, but the source does not show independent factual verification, a deterministic scoring rubric, retry policy or model fallback. A syntactically valid response can still contain vague, unsupported or misleading advice.

6.4 Responsible AI considerations

- A match score is an AI estimate, not an objective hiring result.
- Output quality depends on completeness and accuracy of supplied inputs.
- Candidates should verify technical claims and personalize suggested answers.
- Avoid submitting third-party confidential data without permission.
- Test for biased assumptions, fabricated recommendations and unsafe output.

For academic presentation, describe the output as AI-assisted; do not claim guaranteed accuracy or job placement.

CHAPTER 6 - API AND SECURITY DESIGN

6.5 API design

METHOD	PATH	ACCESS	PURPOSE
GET	/	Public	Service status JSON.
POST	/api/auth/register	Public	Create an account.
POST	/api/auth/login	Public	Authenticate by email and password.
GET	/api/auth/get-me	Protected	Return current account details.
GET	/api/auth/logout	Session-aware	Clear cookie and blacklist token if

			present.
POST	/api/interview/	Protected	Generate and save report; multipart resume supported.
GET	/api/interview/	Protected	List current user’s reports.
GET	/api/interview/report/:interviewId	Protected	Fetch report scoped to current user.
GET	/api/interview/resume/pdf/:id	Protected	Generate resume PDF from report.
GET	/api/interview/plan/pdf/:id	Protected	Download interview plan PDF.

6.6 Security controls present in source

Passwords are hashed with bcryptjs. Login and registration issue a signed JWT in an HTTP-only cookie; production settings enable secure and SameSite=None for cross-site deployment. Protected routes use middleware to check cookie presence, token blacklist and JWT signature. CORS permits configured frontend origins and Vercel preview hostnames.

6.7 Security work recommended

Use secure secret management, rotate exposed credentials, apply rate limits to authentication and costly AI endpoints, enforce file-content validation, cap request sizes and avoid leaking error details in production. Review ownership checks for every report-derived download route. Restrict CORS to trusted deployments when feasible and plan CSRF defenses compatible with cross-site cookies.

Production links use HTTPS; local development may use HTTP.

CHAPTER 7 - USER INTERFACE AND ACCESSIBILITY

7.1 Main screens

- Home: job-description and profile-input workflow.
- Registration and sign-in: account forms with password visibility toggle.
- Report view: match score, questions, skill gaps and preparation plan.

- History: summaries of saved reports for the signed-in user.
- Theme selection: light and dark visual themes.

7.2 Interaction design

The home page introduces a three-step process: add a role, share a profile and receive a game plan. The form labels the job description as required and presents resume upload and self-description as alternate profile methods. Upload guidance states PDF or DOCX and a 3 MB limit; because the backend path uses pdf-parse, DOCX support should be confirmed or removed from UI copy.

7.3 Accessibility considerations

- Use explicit labels, semantic buttons and meaningful heading order.
- Password visibility has an accessible Show / Hide name.
- Provide keyboard-visible focus outlines and keyboard-operable controls.
- Make errors and status perceivable without relying on color alone.
- Check contrast in both themes, zoom, narrow screens and screen readers.

7.4 Visual consistency

Theme tokens and SCSS feature styles support consistent surfaces, colors and responsive layout. Theme choice is stored in browser local storage. Visual QA should test sign-in, registration, upload and report screens in both themes and at mobile and desktop widths.

USABILITY TEST: Ask representative students to complete the workflow without coaching; record completion, errors and suggestions.

FIGURES 1 AND 2 - HOME SCREENSHOTS

The following screenshots were captured from the publicly deployed application on 28 October 2026. They document the home-page workflow in the dark and light appearance settings. User information and report content are not shown.

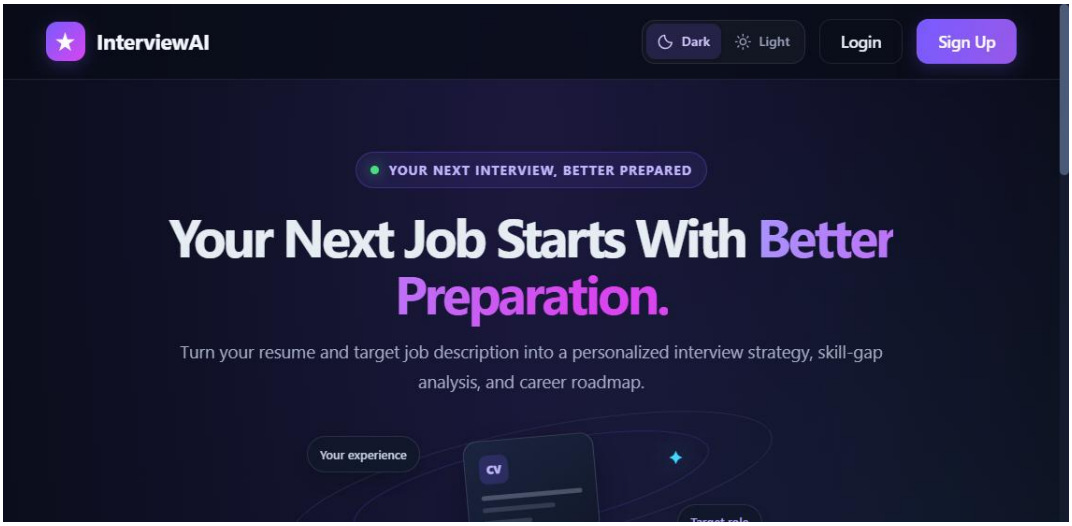


Figure 1. Resume Analyser home page in dark theme.

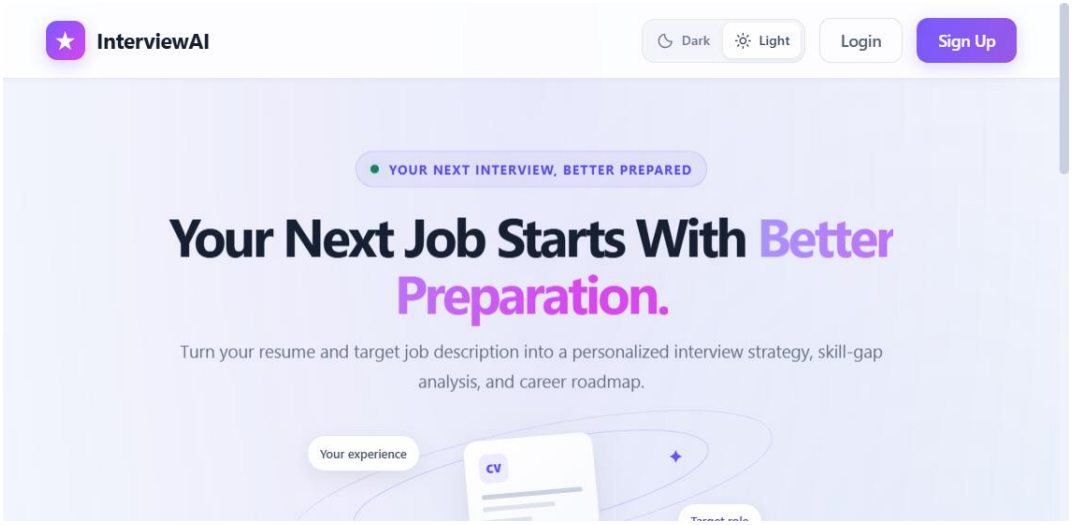


Figure 2. Resume Analyser home page in light theme.

SOURCE: Public deployment: https://mini-project-six-black.vercel.app/

FIGURES 3 AND 4 - AUTHENTICATION SCREENSHOTS

These screenshots show the deployed sign-in and account-creation interfaces. The password visibility control is presented as an eye icon with an accessible text label. No credentials were entered during capture.

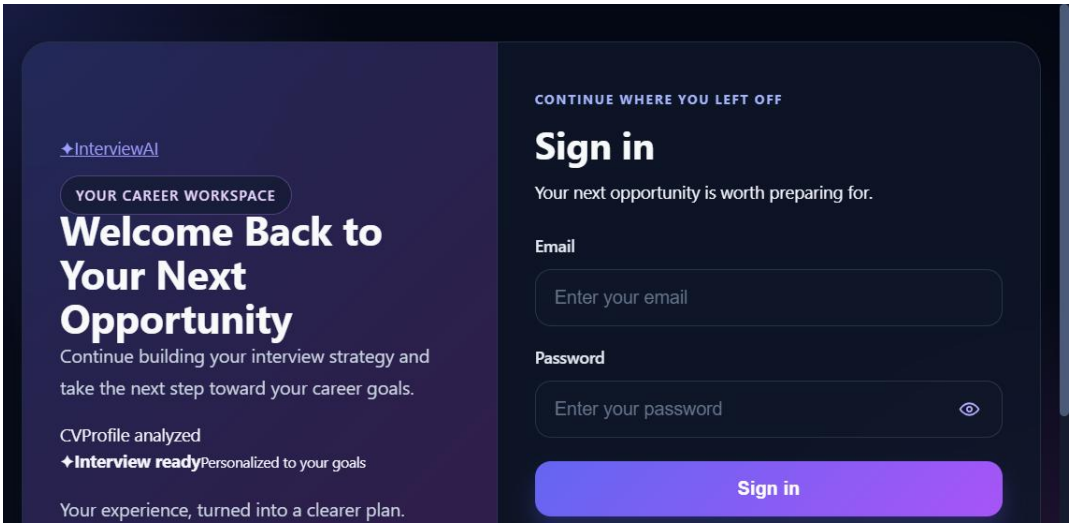


Figure 3. Sign-in page with email and password fields.

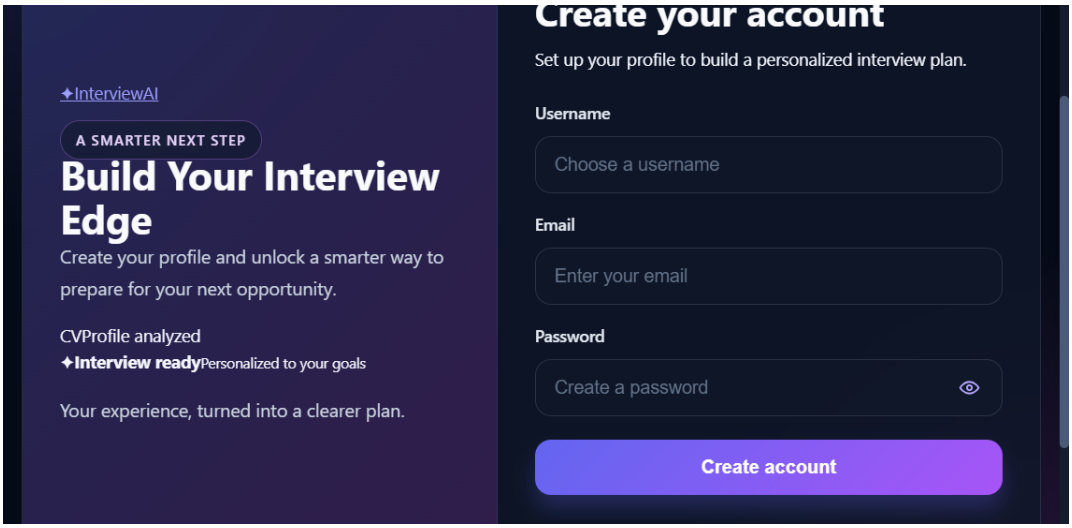


Figure 4. Account-creation page with username, email and password fields.

PRIVACY: Screenshots contain no real account credentials or private user records.

CHAPTER 8 - TESTING AND QUALITY PLAN

8.1 Verification status

The inspected frontend package defines build and lint scripts but no dedicated test script. A production build passed during project work; this confirms bundling, not correctness of every functional workflow. The cases below form a recommended manual/API plan and should be executed in a controlled environment before final demonstration. Record actual outcomes rather than marking unexecuted tests as passed.

ID	TEST SCENARIO	EXPECTED RESULT
T-01	Register with valid unique data.	Account created; session cookie issued.
T-02	Register duplicate account.	Clear error; no duplicate account.
T-03	Login with invalid password.	Authentication rejected.
T-04	Request report without job description.	HTTP 400 with useful message.
T-05	Request report without profile source.	HTTP 400 with useful message.
T-06	Upload valid PDF within limit.	Text extracted; structured report generated.
T-07	Upload oversized or unsupported file.	Request rejected with explicit error.
T-08	Access report without valid cookie.	HTTP 401; no private data returned.
T-09	Attempt another account’s report ID.	Access denied or not found.
T-10	Download plan PDF.	Valid PDF attachment opens.
T-11	Logout and reuse old token.	Blacklisted token rejected.
T-12	Call API from trusted/untrusted origins.	CORS matches configured policy.

8.2 Quality measurements

Record build status, test pass rate, report-schema parse rate, median generation time over a stated sample, download success rate and user-rated usefulness. Do not invent benchmark values: capture the environment, date, sample size and observed results.

CHAPTER 8 - DEPLOYMENT AND OPERATIONS

8.3 Deployment topology

COMPONENT	DEPLOYMENT	CONFIGURATION / RESPONSIBILITY
Frontend	Vercel	Static Vite build; production API URL via VITE_API_URL.
Backend	Render	Node.js Express process; status endpoint and API routes.
Database	MongoDB service	Connection string supplied via MONGO_URI.
AI provider	Google Gemini API	Server credential supplied via GOOGLE_GENAI_API_KEY.

8.4 Runtime configuration

The backend reads environment configuration for database connectivity, JWT signing, Gemini access, server port and runtime environment. Secret values must remain in deployment settings and local environment files, not source control. The frontend embeds only the public API base URL during build; Vite variables prefixed with VITE_ are visible to browser users and must never contain secrets.

8.5 Operational checks

- Check the root status route after deployment.
- Review Render logs for startup, AI, database and PDF errors without logging sensitive user content.
- Check Vercel logs and confirm correct Git branch and environment variables.
- Monitor API quota, database availability, latency, memory and Puppeteer resource use.
- Document rollback, backups, retention and incident-handling procedures.

8.6 Deployment risks

Cross-origin cookies require a compatible CORS allowlist, credentials setting, HTTPS origin and SameSite/Secure configuration. A stale deployment can make source changes appear ineffective; verify the active Vercel commit and build timestamp when debugging UI changes.

CHAPTER 9 - RESULTS AND DISCUSSION

9.1 Implemented outcomes

- A live browser application with registration, sign-in and theme selection.
- A job-description plus resume/self-description entry flow.
- Authenticated report generation and report retrieval routes.
- AI structure for score, questions, gaps and daily tasks.
- MongoDB persistence for accounts and reports.
- PDF routes for generated resume and interview plan.

9.2 Deployment verification

At report preparation, the published frontend is available at https://mini-project-six-black.vercel.app/ and the backend status endpoint responded with JSON status “ok” during a prior check. These observations confirm service availability at the time checked; they do not prove correctness of all API flows or AI results.

9.3 Interpretation

The project demonstrates end-to-end integration of a React client, Express API, MongoDB models and hosted AI model. Its central value is transformation of unstructured profile and role text into a structured preparation artifact. Because model calls are probabilistic, correctness should be evaluated using reviewed samples rather than inferred from a successful HTTP response.

9.4 Evidence to attach during final submission

- Screenshots of home page, sign-in, generated report and PDF output.
- Dated test table with input conditions and actual result.
- Deployment dashboard or commit evidence if required by the institution.
- A sample report generated using a non-sensitive test profile.

If additional tests are conducted, replace this narrative with measured results and distinguish observed outcomes from planned tests.

CHAPTER 9 - LIMITATIONS AND FUTURE SCOPE

9.5 Current limitations

- AI output may be incomplete, inconsistent or factually incorrect.

- Match score has no documented calibrated dataset or validated rubric.
- DOCX is advertised, while the inspected parser is PDF-oriented.
- No dedicated automated frontend test script is in the inspected manifest.
- Rate limiting and detailed observability are not evidenced in inspected source.
- Resume and plan PDF flows need edge-case testing.

9.6 Short-term future scope

A first improvement cycle should prioritize reliable DOCX extraction, server-side file-type validation and clearer upload messaging. The project should add repeatable unit and API integration tests, along with browser end-to-end tests for registration, report generation, history access and downloads. CI should run these checks for each change.

9.7 Medium-term future scope

The report could show which parts of the candidate profile support each recommendation, distinguish user-provided evidence from model inference, and provide controls to regenerate or edit a preparation plan. A reviewed evaluation set could assess answer relevance, score consistency and bias. The product could add data-retention controls, account deletion, rate limits, usage monitoring and structured privacy notices.

9.8 Long-term future scope

Possible extensions include progress tracking across preparation days, user feedback on question usefulness, multilingual support, configurable interview formats and optional integrations with calendars or learning resources. Such features should be introduced only after security, privacy, accessibility and model-quality evaluation have been strengthened.

Recommended order: improve correctness, privacy and testability before adding more AI features.

CHAPTER 10 - CONCLUSION

10.1 Conclusion

Resume Analyser is a full-stack mini project that applies generative AI to a practical career-preparation workflow. It combines a target job description with candidate context, requests structured interview guidance, persists user reports and supports document export. The implementation demonstrates the integration of a React frontend, Express API, MongoDB data models, cookie-based authentication, file processing and the Gemini API.

The project achieves its main demonstration objective: it turns unstructured job and profile context into a structured preparation artifact with questions, potential skill gaps and a plan. The deployed application and captured interface screenshots provide evidence of the primary user-facing surfaces. The quality of recommendations must still be reviewed by users and tested systematically.

Important work remains in file compatibility, measured AI evaluation, automated testing, privacy controls, operational monitoring and authorization review for every report-derived route. Addressing these areas would make the application more reliable and suitable for broader use.

10.2 Final takeaway

AI can help organize interview preparation, but it does not replace candidate judgment, factual verification or professional career advice.

REFERENCES

The following project resources and primary technical documentation were consulted for this report. Web references were accessed on 28 October 2026; URLs may change.

NO.	REFERENCE	URL / SOURCE
1	Resume Analyser project repository	https://github.com/myArpit/mini_project
2	Live web application	https://mini-project-six-black.vercel.app/
3	Google Gemini API documentation	https://ai.google.dev/gemini-api/docs
4	React documentation	https://react.dev/
5	Vite documentation	https://vite.dev/guide/
6	Express documentation	https://expressjs.com/
7	Node.js documentation	https://nodejs.org/docs/
8	MongoDB documentation	https://www.mongodb.com/docs/
9	Mongoose documentation	https://mongoosejs.com/docs/
10	Zod documentation	https://zod.dev/
11	Puppeteer documentation	https://pptr.dev/
12	Vercel documentation	https://vercel.com/docs
13	Render documentation	https://render.com/docs
14	OWASP Authentication Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
15	OWASP File Upload Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

Source files reviewed

Backend/src/app.js; Backend/src/routes/auth.routes.js; Backend/src/routes/interview.routes.js; Backend/src/controllers/auth.controller.js; Backend/src/controllers/interview.controller.js; Backend/src/middlewares/auth.middleware.js;

Backend/src/middlewares/file.middleware.js; Backend/src/models/user.model.js; Backend/src/models/interviewReport.model.js; Backend/src/services/ai.service.js; Frontend/src/features/auth; Frontend/src/features/interview; Frontend/package.json; Backend/package.json.

Use the institution's required citation style if it differs from this concise numbered reference list.

APPENDIX A - TECHNICAL SETUP AND CONFIGURATION

A.1 Prerequisites

- Node.js and npm compatible with the project manifests.
- A MongoDB database and connection string.
- A Google AI Studio / Gemini API credential.
- Git and a modern browser for development and demonstration.

A.2 Local run outline

TASK	COMMAND / ACTION
Install backend packages	cd Backend; npm install
Configure backend	Create Backend/.env locally; set required server values.
Start backend	cd Backend; npm run dev (development) or npm start.
Install frontend	cd Frontend; npm install
Set API base URL	Create Frontend/.env.local with VITE_API_URL=http://localhost:3000 if needed.
Start frontend	cd Frontend; npm run dev
Production build	cd Frontend; npm run build
Quality check	cd Frontend; npm run lint

A.3 Configuration variable inventory

VARIABLE	PURPOSE	SECRET?
PORT	Backend listen port; deployment may assign this	No

	value.	
MONGO_URI	MongoDB connection string.	Yes
JWT_SECRET	Secret used to sign and verify JWTs.	Yes
GOOGLE_GENAI_API_KEY	Credential for Gemini API calls.	Yes
NODE_ENV	Controls production-specific behavior.	No
VITE_API_URL	Public frontend API URL compiled into browser assets.	No

Never paste real credentials into this report, Git, screenshots or public build variables. Use deployment secret settings and rotate exposed credentials.

APPENDIX B - API REFERENCE AND RESPONSE SCHEMA

B.1 AI report schema summary

SECTION	CONTENT SHAPE
Title	String naming the target role/report.
Match score	Number in the configured 0-100 range.
Technical questions	Array of question, intention and answer guidance.
Behavioral questions	Array of question, intention and answer guidance.
Skill gaps	Array of skill and severity: low, medium or high.
Preparation plan	Array of day, focus and task list.

B.2 Response and status conventions

- Successful registration: HTTP 201; successful login: HTTP 200.
- Successful report generation: HTTP 201 with a saved report.
- Missing report: HTTP 404; missing/invalid token: HTTP 401.
- Invalid required report input: HTTP 400.
- Unhandled generation errors: HTTP 500 response.

B.3 Glossary

TERM	DEFINITION
Generative AI	A model that creates text or other output from input context.
JWT	A signed token carrying claims used to identify a session subject.
Multer	Express middleware used here to receive uploaded files in memory.
Mongoose	Object-document modeling library used with MongoDB.
CORS	Browser policy controlling which origins may read cross-origin responses.
Schema	Definition of expected data shape and constraints.
Puppeteer	Browser automation library used to render HTML into PDF.
HTTP-only cookie	A cookie unavailable to ordinary client-side JavaScript.

Submission checklist: fill student and college placeholders, attach actual test evidence, verify deployed URLs and upload behavior, and follow departmental formatting and binding rules.